

Corporate Technical, Functional & Operations Documentation

Project: QuizForge (Fourise Quiz Hub)

1. Executive Summary

1.1 What is this project?

QuizForge (also referenced as **Fourise Quiz Hub**) is an enterprise-grade, real-time multiplayer quiz engine and interactive learning management platform inspired by Kahoot. The platform provides real-time gamified question streaming, synchronized multi-player lobby orchestration, base64 QR code and 6-digit numeric PIN session access, dynamic time-weighted scoring calculations, and extensive post-game analytical export utilities (.xlsx, .pdf, .docx).

1.2 Purpose & Business Objective

The primary business objective of QuizForge is to provide educators, enterprise trainers, corporate seminar hosts, and event organizers with a robust, zero-friction, scalable platform for engaging audiences. The system eliminates registration barriers for participants—allowing instant game entry via QR code or PIN—while empowering hosts with deep analytics, customizable question banks, custom background themes, and exported compliance/performance reports.

1.3 Main Features

- **Stateless JWT Authentication & Password Recovery:** HMAC SHA256 token authorization backed by SMTP email password reset and security question verification fallbacks.
- **Interactive Quiz Builder & Authoring Suite:** Single-choice 4-option quiz creation with custom time limits (default 60s), category tagging, background image picker, and full CRUD management.
- **Real-time Engine & Socket Orchestration:** WebSockets powered by Socket.IO managing real-time host lobbies, synchronized timers, instant answer processing, score streaks, and live rankings.
- **Frictionless QR Code & 6-Digit PIN Join Engine:** Automatic numeric PIN generation and base64 QR Code rendering for instant mobile onboarding.
- **Scoring & Performance Engine:** Time-decay scoring matrix evaluating correct choices and answer response speeds.
- **Reporting & Export Suite:** Client-side document generation outputting Excel workbooks (`.xlsx`), PDF reports (`.pdf`), and Word documents (`.docx`).
- **Responsive UI & Dynamic Theme Switcher:** Framer Motion route transitions, customizable visual themes, particle confetti celebrations, and Tailwind CSS responsive views.

1.4 Users & Target Audience

1. **Quiz Hosts / Educators / Enterprise Trainers:** Registered users who design quizzes, launch live sessions, orchestrate real-time game flows, and analyze post-game analytics.
2. **Players / Participants / Students:** Guest users who join sessions via PIN/QR code on mobile or desktop without requiring account creation.
3. **System Administrators & DevOps Engineers:** Maintainers responsible for managing backend Node services, MongoDB databases, WebSocket connections, and deployment pipelines.

1.5 Overall Architecture Overview

QuizForge uses a decoupled Client-Server Architecture:

- **Frontend SPA Layer:** Built with React 19, Vite 8, React Router DOM v7, TanStack React Query v5, and Socket.IO-Client.
- **Backend API & WebSocket Server:** Powered by Node.js, Express 5, and Socket.IO.

- **Database & Storage Layer:** MongoDB managed via Mongoose 9 schemas (with `mongodb-memory-server` fallback for offline local development).
-

2. Technology Stack

2.1 Core Stack Breakdown

Frontend Stack

- **UI Library:** [React 19.2.6](#)
- **Build System & Dev Server:** [Vite 8.0.12](#)
- **Routing:** [React Router DOM 7.18.1](#)
- **Async & Server State:** [TanStack React Query 5.101.2](#)
- **HTTP Client:** [Axios 1.18.1](#)
- **Realtime Socket Client:** [Socket.IO Client 4.8.3](#)
- **Styling Framework:** [Tailwind CSS 3.4.19](#) + [PostCSS](#) & [Autoprefixer](#)
- **Animation Libraries:** [Framer Motion 12.42.1](#), [Canvas Confetti 1.9.4](#)
- **Iconography:** [Lucide React 1.22.0](#)
- **Forms & Validation:** [React Hook Form 7.80.0](#)
- **Toast Notifications:** [React Hot Toast 2.6.0](#)
- **Export Generators:** [XLSX 0.18.5](#), [jsPDF 4.2.1](#), [jspdf-autotable 5.0.8](#), [docx 9.7.1](#)
- **Scanner Utility:** [HTML5-QRCode 2.3.8](#)

Backend Stack

- **Runtime:** Node.js (v18+ recommended)
 - **Framework:** [Express 5.2.1](#) (CommonJS module system)
 - **Database Driver / ORM:** [Mongoose 9.7.3](#) / [MongoDB 7.4.0](#)
 - **Development Database:** [mongodb-memory-server 11.2.0](#)
 - **Realtime Protocol Server:** [Socket.IO Server 4.8.3](#)
 - **Security & Auth:** [jsonwebtoken 9.0.3](#), [bcryptjs 3.0.3](#), [cors 2.8.6](#)
 - **Email Service:** [Nodemailer 9.0.3](#)
 - **QR Code Rendering Engine:** [QRCode 1.5.4](#)
 - **Environment Parser:** [dotenv 17.4.2](#)
-

3. Folder Structure

3.1 Workspace Folder Tree

```

QuizForge/
├── .gitignore           # Root Git ignore declarations
├── package.json         # Root monorepo orchestration & concurrently scripts
├── package-lock.json    # Root package lock
├── README.md           # Root repository documentation
├── docs/               # Corporate documentation & handoff manuals
│   ├── HANDOFF.md      # Production deployment & handoff guide
│   └── Corporate_Documentation.md # Existing architecture reference manual
├── postman/            # API Postman collection exports
├── client/             # React 19 Single Page Application
│   ├── index.html      # Root HTML entry point
│   ├── package.json    # Client dependencies & scripts
│   ├── package-lock.json # Client lock file
│   ├── postcss.config.js # PostCSS tailwind integration
│   ├── tailwind.config.js # Tailwind custom colors & design tokens
│   ├── vite.config.js   # Vite configuration & proxy definitions
│   ├── vercel.json      # Vercel deployment routing rules
│   ├── .env.example     # Client environment template
│   └── public/          # Static assets (avatars, icons)
│       └── src/
│           ├── App.css      # Global styles
│           ├── App.jsx      # Route definitions & provider wrappers
│           ├── index.css    # Tailwind entry point
│           ├── main.jsx     # React 19 mount point
│           └── api/         # Axios base configuration
│               └── config.js # Central API URL resolver
│           ├── components/  # Reusable frontend components
│               ├── AnimatedPage.jsx # Framer motion transition wrapper
│               ├── Avatar.jsx      # Avatar selection & render component
│               ├── BackgroundPicker.jsx # Quiz artwork selection picker
│               ├── DemoModal.jsx   # Interactive system demo modal
│               ├── Footer.jsx      # Application footer
│               ├── Logo.jsx        # QuizForge branding logo
│               ├── Navbar.jsx      # Navigation header bar
│               ├── PlanComparisonTable.jsx # Pricing comparison grid
│               ├── QuickRegisterModal.jsx # Quick signup modal dialog
│               ├── ReportsModal.jsx # PDF / Excel report generator modal
│               ├── Sidebar.jsx     # Host dashboard navigation sidebar
│               ├── ThemeBackground.jsx # Dynamic theme container
│               ├── ThemeSelector.jsx # Global visual theme selector
│               └── landing/        # Public landing page sections (11 files)
│           ├── context/        # React Context providers
│               ├── GameContext.jsx # Live quiz state provider
│               └── ThemeContext.jsx # Theme state provider
│           └── pages/          # SPA Page Views (17 files)
│               ├── AnswerResult.jsx # Live answer feedback screen
│               ├── CreateQuiz.jsx   # Interactive quiz authoring screen
│               ├── Dashboard.jsx    # Host control center
│               ├── EditQuiz.jsx     # Quiz editor view
│               ├── FinalResult.jsx  # Winner podium screen
│               ├── ForgotPassword.jsx # Multi-step account recovery
│               ├── HostLobby.jsx    # Live host lobby & PIN screen
│               ├── JoinGame.jsx     # Player PIN entry screen
│               ├── LandingPage.jsx  # Public landing page
│               ├── Leaderboard.jsx  # Interim game leaderboard
│               ├── LiveQuiz.jsx     # Live player question screen
│               ├── Login.jsx        # Account login view
│               ├── MyQuizzes.jsx    # Host quiz repository view
│               ├── Register.jsx     # Account sign-up view
│               └── ResetPassword.jsx # Token password reset view

```

```

├── ResultsAnalytics.jsx # Analytics breakdown screen
├── WaitingRoom.jsx     # Player lobby staging screen
├── services/           # Backend API service bridges (7 files)
│   ├── api.js          # Axios instance definition
│   ├── authService.js  # Authentication API calls
│   ├── gameService.js  # Session API calls
│   ├── infoService.js  # Plans & FAQ API calls
│   ├── quizService.js  # Quiz CRUD API calls
│   ├── resultService.js # Analytics API calls
│   └── socketService.js # Socket.IO client event bridge
├── themes/             # Theme tokens & design definitions
└── server/             # Express 5 & Socket.IO Backend
    ├── package.json    # Backend dependencies & scripts
    ├── package-lock.json # Backend lock file
    ├── index.js        # Active Express 5 server entry point
    ├── server.js       # Legacy/alternative server entry point
    ├── .env            # Backend local environment secrets
    ├── .env.example    # Environment variable template
    ├── config/         # Configuration modules
    │   └── db.js       # Mongoose MongoDB connection initializer
    ├── controllers/    # Express route controllers (5 files)
    │   ├── authController.js # Auth & password reset logic
    │   ├── gameController.js # Game PIN & session logic
    │   ├── infoController.js # Public FAQs & plans logic
    │   ├── quizController.js # Quiz CRUD logic
    │   └── resultController.js # Analytics aggregation logic
    ├── middleware/     # Express middleware
    │   └── authMiddleware.js # JWT verification middleware
    ├── models/         # Mongoose data models (6 files)
    │   ├── FAQ.js      # FAQ document schema
    │   ├── GameSession.js # Active session schema
    │   ├── Plan.js     # Subscription tier schema
    │   ├── Quiz.js     # Quiz bank schema
    │   ├── Result.js   # Archived session schema
    │   └── User.js     # User account schema
    ├── routes/         # Express API router declarations (6 files)
    │   ├── authRoutes.js # Auth routes
    │   ├── gameRoutes.js # Game routes
    │   ├── imageRoutes.js # Image proxy routes
    │   ├── infoRoutes.js # Info routes
    │   ├── quizRoutes.js # Quiz routes
    │   └── resultRoutes.js # Result routes
    ├── socket/         # Realtime Socket handlers
    │   └── index.js    # Socket.IO room & timer orchestration
    ├── tests/          # Automated backend unit tests
    │   ├── auth.test.js # Auth unit test suite
    │   └── serverStartup.test.js # Server startup verification
    └── utils/          # Server helper utilities
        └── sendEmail.js # SMTP email delivery utility

```

4. Git Information

- **Repository Layout:** Monorepo architecture housing both `client/` and `server/` sub-packages under a unified root directory.
- **Root Entry point:** `package.json` orchestrating concurrently execution via `npm run dev:full`.
- **Git Exclusions (`.gitignore`):**
 - Excludes `node_modules/` across all directories.
 - Excludes active `.env` configuration files.
 - Excludes compiled distribution artifacts (`client/dist`).

QuizForge Monorepo (Root)

```

|
├─ client/  → Single Page Application (Deployed to Vercel / Netlify)
├─ server/  → Node/Express API + WebSocket Server (Deployed to Render / Railway / VPS)

```

5. Environment Variables

5.1 Backend Environment Variables (server/.env)

Variable Name	Required	Default Value	File Consumption	Description
PORT	Optional	5000	server/index.js:7	Port number for Express server.
HOST	Optional	0.0.0.0	server/index.js:8	Server bind network interface address.
MONGO_URI	Required	mongodb://127.0.0.1:27017/quizforge	server/config/db.js:6	MongoDB Atlas / local MongoDB connection string.
JWT_SECRET	Required	fallback_secret_key	server/controllers/authController.js:14	Secret key for signing/verifying JWTs.
JWT_EXPIRE	Optional	7d	server/controllers/authController.js:15	Expiration timeframe for authentication tokens.
FRONTEND_URL	Optional	http://localhost:5173	server/index.js:46	Origin URL used for CORS validation and email reset links.
EMAIL_HOST	Optional	N/A	server/utis/sendEmail.js:6	SMTP Mail host server address.
EMAIL_PORT	Optional	587	server/utis/sendEmail.js:7	SMTP Mail host port.
EMAIL_USER	Optional	N/A	server/utis/sendEmail.js:9	SMTP authentication username.
EMAIL_PASS	Optional	N/A	server/utis/sendEmail.js:10	SMTP authentication password.

5.2 Frontend Environment Variables (client/.env.example)

Variable Name	Required	Default Value	File Consumption	Description
VITE_API_URL	Recommended	Automatically derived	client/src/api/config.js:3	Base HTTP URL for backend REST API endpoints.

6. Database Documentation

QuizForge uses **MongoDB** as its primary document store, accessed through **Mongoose 9** schemas.

6.1 Mongoose Collections & Schemas

1. User Schema (server/models/User.js)

- Collection: users

- **Purpose:** Stores host user credentials, security questions, and password reset tokens.

- **Fields:**

- `name` (String, Required, Trimmed)
- `email` (String, Required, Unique, Lowercase, Trimmed)
- `password` (String, Required, Hashed via bcryptjs)
- `securityQuestion` (String, Optional)
- `securityAnswer` (String, Optional, Hashed via bcryptjs)
- `resetPasswordToken` (String, Optional)
- `resetPasswordExpire` (Date, Optional)
- `createdAt` (Date, Default: `Date.now`)

2. Quiz Schema (`server/models/Quiz.js`)

- **Collection:** `quizzes`

- **Purpose:** Stores authored quiz templates, embedded questions, options, and correctness keys.

- **Fields:**

- `title` (String, Required, Trimmed)
- `description` (String, Optional)
- `category` (String, Default: `'General'`)
- `createdBy` (ObjectId `ref: 'User'`, Required)
- `isPublic` (Boolean, Default: `true`)
- `questions` (Array of Subdocuments):
 - `questionText` (String, Required)
 - `options` (Array of Strings, Required, Exactly 4 options)
 - `correctAnswerIndex` (Number, Required, 0-3)
 - `timeLimit` (Number, Default: 60 seconds)
 - `imageUrl` (String, Optional)
- `createdAt` / `updatedAt` (Timestamps: `true`)

3. GameSession Schema (`server/models/GameSession.js`)

- **Collection:** `gamesessions`

- **Purpose:** Tracks active live game instances, PINs, player rosters, and live answer submissions.

- **Fields:**

- `pin` (String, Required, Unique, 6-digit numeric string)
- `quizId` (ObjectId `ref: 'Quiz'`, Required)
- `hostId` (ObjectId `ref: 'User'`, Required)
- `qrCodeUrl` (String, Base64 QR Code string)
- `status` (String, Enum: `['waiting', 'active', 'completed']`, Default: `'waiting'`)
- `currentQuestionIndex` (Number, Default: 0)
- `players` (Array of Subdocuments):
 - `socketId` (String)
 - `name` (String, Trimmed)
 - `avatar` (String)
 - `score` (Number, Default: 0)
 - `answers` (Array): `{ questionIndex: Number, selectedOption: Number, isCorrect: Boolean, scoreEarned: Number, responseTimeSec: Number }`
- `createdAt` (Date, Default: `Date.now`, Expiration index enabled)

4. Result Schema (`server/models/Result.js`)

- **Collection:** `results`

- **Purpose:** Archives post-game performance summaries for historical reporting.

- **Fields:**

- `sessionId` (String/ObjectId, Required)

- `quizId` (ObjectId `ref: 'Quiz'`, Required)
- `quizTitle` (String, Required)
- `hostId` (ObjectId `ref: 'User'`, Required)
- `totalQuestions` (Number, Required)
- `playersCount` (Number, Required)
- `players` (Array of Summarized Player Results)
- `winner` (Object: `{ name: String, score: Number, avatar: String }`)
- `completedAt` (Date, Default: `Date.now`)

5. Plan Schema (`server/models/Plan.js`)

- **Collection:** `plans`
- **Purpose:** Stores subscription tier options displayed on the landing page.
- **Fields:** `name`, `price`, `billingCycle`, `features` (Array), `popular` (Boolean), `isActive` (Boolean)

6. FAQ Schema (`server/models/FAQ.js`)

- **Collection:** `faqs`
- **Purpose:** Stores system help items and dynamic FAQ content.
- **Fields:** `question`, `answer`, `category`, `displayOrder`, `isActive`

7. Database Collection Mapping

Collection Name	Schema File	Controllers Reading	Controllers Writing	Routes Exposing	Frontend Consumers
<code>users</code>	<code>User.js</code>	<code>authController.js</code>	<code>authController.js</code>	<code>/api/auth/*</code>	<code>Login.jsx</code> , <code>Register.jsx</code> , <code>ForgotPassword.jsx</code>
<code>quizzes</code>	<code>Quiz.js</code>	<code>quizController.js</code> , <code>gameController.js</code>	<code>quizController.js</code>	<code>/api/quiz/*</code>	<code>Dashboard.jsx</code> , <code>CreateQuiz.jsx</code> , <code>EditQuiz.jsx</code> , <code>MyQuizzes.jsx</code>
<code>gamesessions</code>	<code>GameSession.js</code>	<code>gameController.js</code> , <code>socket/index.js</code>	<code>gameController.js</code> , <code>socket/index.js</code>	<code>/api/game/*</code>	<code>HostLobby.jsx</code> , <code>JoinGame.jsx</code> , <code>LiveQuiz.jsx</code> , <code>AnswerResult.jsx</code>
<code>results</code>	<code>Result.js</code>	<code>resultController.js</code>	<code>resultController.js</code>	<code>/api/result/*</code>	<code>ResultsAnalytics.jsx</code> , <code>Dashboard.jsx</code> , <code>ReportsModal.jsx</code>
<code>plans</code>	<code>Plan.js</code>	<code>infoController.js</code>	Seed / Admin	<code>/api/info/plans</code>	<code>LandingPage.jsx</code> , <code>PricingSection.jsx</code>
<code>faqs</code>	<code>FAQ.js</code>	<code>infoController.js</code>	Seed / Admin	<code>/api/info/faqs</code>	<code>LandingPage.jsx</code> , <code>FAQSection.jsx</code>

8. Database Flow Diagram

flowchart TD

```
UserReg[User Registration / Login] -->|Creates/Verifies| UserColl[(users Collection)]
UserColl -->|Auth Token Issued| Host[Host Session]
Host -->|Authors Quiz| QuizColl[(quizzes Collection)]
Host -->|Launches Live Session| GameColl[(gamesessions Collection)]
GameColl -->|Reads Questions| QuizColl
Player[Player Device] -->|Joins via PIN & Submits Answers| GameColl
Host -->|Ends Game| ResultColl[(results Collection)]
ResultColl -->|Archived Data| GameColl
Host -->|Views Analytics & Exports| Reports[Excel / PDF Reports]
```

9. API Documentation

9.1 Authentication Endpoints (/api/auth)

POST /api/auth/register

- **Purpose:** Registers a new host user account.
- **Auth:** None (Public)
- **Request Body:** { "name": "John", "email": "john@example.com", "password": "SecretPassword123", "securityQuestion": "Pet Name", "securityAnswer": "Fluffy" }
- **Response Status:** 201 Created
- **Response Payload:** { "token": "JWT_STRING", "user": { "id": "...", "name": "John", "email": "..." } }

POST /api/auth/login

- **Purpose:** Authenticates host credentials and returns JWT session token.
- **Auth:** None (Public)
- **Request Body:** { "email": "john@example.com", "password": "SecretPassword123" }
- **Response Status:** 200 OK
- **Response Payload:** { "token": "JWT_STRING", "user": { "id": "...", "name": "John", "email": "..." } }

GET /api/auth/profile

- **Purpose:** Fetches currently authenticated user details.
- **Auth:** Bearer Token (authMiddleware.js)
- **Response Status:** 200 OK
- **Response Payload:** { "user": { "id": "...", "name": "John", "email": "..." } }

POST /api/auth/forgot-password

- **Purpose:** Initiates password reset via email or fetches security question.
- **Request Body:** { "email": "john@example.com" }

POST /api/auth/verify-security-answer

- **Purpose:** Verifies security answer and issues reset token if email service is unavailable.
- **Request Body:** { "email": "john@example.com", "securityAnswer": "Fluffy" }

POST /api/auth/reset-password

- **Purpose:** Resets user password using reset token.
- **Request Body:** { "token": "RESET_TOKEN", "newPassword": "NewPassword123" }

9.2 Quiz Endpoints (/api/quiz)

GET /api/quiz/list

- **Purpose:** Returns list of public active quizzes.
- **Auth:** None (Public)

GET /api/quiz/user/myquizzes

- **Purpose:** Returns quizzes authored by the authenticated caller.
- **Auth:** Bearer Token

GET /api/quiz/:id

- **Purpose:** Retrieves a single quiz by ID.
- **Auth:** None (Public)

POST /api/quiz/create

- **Purpose:** Saves a new quiz document.
- **Auth:** Bearer Token
- **Request Body:** { "title": "Math Quiz", "category": "Education", "questions": [...] }

PUT /api/quiz/:id

- **Purpose:** Updates an existing quiz.
- **Auth:** Bearer Token (Owner check)

DELETE /api/quiz/:id

- **Purpose:** Deletes an owned quiz.
- **Auth:** Bearer Token (Owner check)

9.3 Game Session Endpoints (/api/game)

POST /api/game/create

- **Purpose:** Creates active `GameSession`, generates 6-digit numeric PIN and QR code.
- **Auth:** Bearer Token
- **Request Body:** { "quizId": "QUIZ_OBJECT_ID" }
- **Response Payload:** { "session": { "pin": "849201", "qrCodeUrl": "data:image/png;base64,...", "quizId": "..." } }

POST /api/game/join

- **Purpose:** Registers player participant into session by PIN.
- **Auth:** None (Public)
- **Request Body:** { "pin": "849201", "name": "Alex", "avatar": "avatar1.png" }

GET /api/game/:pin

- **Purpose:** Retrieves current session state by PIN.
- **Auth:** None (Public)

POST /api/game/answer

- **Purpose:** Processes player answer submission and calculates score points.
- **Auth:** None (Public)
- **Request Body:** { "pin": "849201", "playerName": "Alex", "questionIndex": 0, "selectedOption": 2, "responseTimeSec": 4.5 }

9.4 Analytics & Results Endpoints (/api/result)

POST /api/result/save

- **Purpose:** Archives finished session results to `Result` collection.

- **Auth:** Bearer Token

GET /api/result/my

- **Purpose:** Retrieves past session analytics list for host.
- **Auth:** Bearer Token

GET /api/result/:sessionId

- **Purpose:** Returns comprehensive analytics report for a specific session ID.
 - **Auth:** Bearer Token
-

10. Frontend Documentation

10.1 React SPA Architecture

The frontend client is constructed as a modern component-driven Single Page Application built on **React 19** and **Vite 8**.

- **Root Router:** Managed in `App.jsx` utilizing `BrowserRouter` and `Routes`.
 - **State Providers:**
 - `ThemeContext` : Provides theme toggle capabilities across pages.
 - `GameContext` : Manages current session PIN, player identity, score totals, and WebSocket listeners.
 - `QueryClientProvider` : Manages server data caching via TanStack React Query.
 - **UI Styling:** Styled with Tailwind CSS classes, supplemented by `framer-motion` page animations (`AnimatedPage.jsx`) and `react-hot-toast` notifications.
-

11. Complete Page Documentation

Page Component	Route Path	Purpose	Key Services / APIs	Access Level
LandingPage.jsx	/	Marketing landing view, demo modal, FAQ, pricing	infoService.js	Public
Login.jsx	/login	Host user authentication	authService.js	Public
Register.jsx	/register	Host user registration	authService.js	Public
ForgotPassword.jsx	/forgot-password	Password recovery flow	authService.js	Public
ResetPassword.jsx	/reset-password/:token	Password reset interface	authService.js	Public
Dashboard.jsx	/dashboard	Host home metrics & quick actions	quizService.js , resultService.js	Authenticated Host
MyQuizzes.jsx	/quiz/my	Manage created quizzes	quizService.js , gameService.js	Authenticated Host
CreateQuiz.jsx	/quiz/create	Interactive quiz authoring suite	quizService.js	Authenticated Host
EditQuiz.jsx	/quiz/edit/:id	Modify existing quiz	quizService.js	Authenticated Host
HostLobby.jsx	/host/lobby/:pin	Host waiting room with PIN & QR Code	gameService.js , socketService.js	Authenticated Host
ResultsAnalytics.jsx	/results/:sessionId	Post-game analytics & export hub	resultService.js	Authenticated Host
JoinGame.jsx	/join	Guest player PIN & avatar entry	gameService.js	Public Player
WaitingRoom.jsx	/waiting/:pin	Player lobby waiting screen	socketService.js	Public Player
LiveQuiz.jsx	/live/:pin	Live player question answer screen	gameService.js , socketService.js	Public Player
AnswerResult.jsx	/result/answer/:pin	Post-question player answer feedback	socketService.js	Public Player
Leaderboard.jsx	/leaderboard/:pin	Interim game rankings	socketService.js	Public Player / Host
FinalResult.jsx	/final-result/:pin	Game winner podium & confetti	resultService.js , socketService.js	Public Player / Host

12. Dashboard Documentation

12.1 Host Control Center (Dashboard.jsx)

The Host Dashboard provides an integrated operational center for quiz creators:

- **Metrics Summary Cards:** Total Quizzes Created, Total Live Games Hosted, Total Participants Engaged, and Average Score Analytics.
- **Quick Game Launch Grid:** Instant "Host Now" trigger on any saved quiz card.
- **Recent Session Activity:** Quick link access to [ResultsAnalytics.jsx](#) reports.
- **Navigation Shortcuts:** Quick actions for creating new quizzes, modifying profile settings, or viewing pricing tier upgrades.

13. Navigation Flow

```
graph TD
    Landing["Landing Page (/)"] -->|Login Click| Login["Login (/login)"]
    Landing -->|Register Click| Register["Register (/register)"]
    Landing -->|Join Game Click| Join["Join Game (/join)"]

    Login -->|Auth Success| Dash["Host Dashboard (/dashboard)"]
    Register -->|Auth Success| Dash

    Dash -->|Create Quiz| CreateQ["Create Quiz (/quiz/create)"]
    Dash -->|My Quizzes| MyQ["My Quizzes (/quiz/my)"]
    Dash -->|View Analytics| Analytics["Analytics (/results/:sessionId)"]

    MyQ -->|Host Game| Lobby["Host Lobby (/host/lobby/:pin)"]
    Join -->|PIN Validated| Waiting["Waiting Room (/waiting/:pin)"]

    Lobby -->|Host Starts Game| LiveQ["Live Quiz (/live/:pin)"]
    Waiting -->|Game Starts Event| LiveQ

    LiveQ -->|Answer Submitted| Feedback["Answer Feedback (/result/answer/:pin)"]
    Feedback -->|Question Ended| InterLdr["Leaderboard (/leaderboard/:pin)"]
    InterLdr -->|Final Question Ended| Podium["Final Podium (/final-result/:pin)"]
    Podium -->|View Full Report| Analytics
```

14. Authentication Flow

14.1 JWT Authentication Mechanism

1. Host enters credentials on `Login.jsx`.
2. Request dispatched to `POST /api/auth/login`.
3. Controller verifies email existence and calls `bcrypt.compare(password, user.password)`.
4. Upon successful comparison, server issues JWT signed with `JWT_SECRET` containing `{ id: user._id }`.
5. Frontend stores token in `localStorage.setItem('token', token)`.
6. Subsequent REST requests append header `Authorization: Bearer <token>`.
7. Backend `authMiddleware.js` intercepts protected routes, validates token payload, and injects `req.user`.

```
sequenceDiagram
    autonumber
    actor Host
    participant Client
    participant Express
    participant MongoDB

    Host->>Client: Submit Credentials
    Client->>Express: POST /api/auth/login
    Express->>MongoDB: User.findOne({ email })
    MongoDB-->>Express: Return User Document
    Express->>Express: bcrypt.compare(inputPassword, user.password)
    Express-->>Client: 200 OK + { token, user }
    Client->>Client: Save token to localStorage
```

15. User Roles & Permission Matrix

Feature / Action	Public / Guest Player	Authenticated Quiz Host	System Admin
Browse Public Landing / FAQs / Plans	Yes	Yes	Yes
Join Live Game via PIN / QR Code	Yes	Yes	Yes
Submit Live Game Answers	Yes	No	Yes
Create / Edit / Delete Quizzes	No	Yes (Owned Quizzes)	Yes (All)
Launch Live Host Session (PIN Gen)	No	Yes	Yes
Access Post-Game Analytics & Export	No	Yes (Owned Sessions)	Yes (All)

16. Module Documentation

16.1 Core Business Modules

- 1. **Authentication Module** (`authController.js`): Handles user registration, JWT generation, password resets, and security question fallbacks.
- 2. **Quiz Management Module** (`quizController.js`): Enables quiz authoring, question validation (4 options required), category selection, and CRUD operations.
- 3. **Realtime Game Engine** (`gameController.js` , `socket/index.js`): Manages random 6-digit numeric PIN generation, QR code encoding, room-based WebSocket subscriptions, question countdown timers, and speed-weighted scoring.
- 4. **Analytics & Reporting Module** (`resultController.js` , `ReportsModal.jsx`): Aggregates session scores, player answer breakdowns, accuracy statistics, and outputs `.xlsx` , `.pdf` , and `.docx` reports.

17. Feature Dependency Mapping

```
Quiz Creation Feature
├─ Component: client/src/pages/CreateQuiz.jsx
├─ Service: client/src/services/quizService.js
├─ Endpoint: POST /api/quiz/create
├─ Middleware: server/middleware/authMiddleware.js
├─ Controller: server/controllers/quizController.js
└─ Model: server/models/Quiz.js

Live Multiplayer Game Feature
├─ Components: HostLobby.jsx, LiveQuiz.jsx, AnswerResult.jsx
├─ Services: gameService.js, socketService.js
├─ Endpoints: POST /api/game/create, POST /api/game/join, POST /api/game/answer
├─ WebSockets: server/socket/index.js (Events: host-start-game, player-join, submit-answer)
└─ Models: GameSession.js, Result.js
```

18. File Dependency Graph

```
client/src/App.jsx
├─ imports pages/LandingPage.jsx
├─ imports pages/Login.jsx
├─ imports pages/Dashboard.jsx
├─ imports pages/CreateQuiz.jsx
├─ imports pages/HostLobby.jsx
├─ imports pages/LiveQuiz.jsx
└─ imports context/GameContext.jsx

server/index.js
├─ imports config/db.js
├─ imports routes/authRoutes.js
├─ imports routes/quizRoutes.js
├─ imports routes/gameRoutes.js
├─ imports routes/resultRoutes.js
└─ imports socket/index.js
```

19. Component Documentation

- **Navbar.jsx** : Global header component displaying branding logo, theme switcher, navigation links, and authenticated host user menu.
- **BackgroundPicker.jsx** : Interactive image selector allowing hosts to pick curated quiz background artwork or upload custom image URLs.
- **ReportsModal.jsx** : Analytics exporter modal rendering triggers to generate Excel (`.xlsx`), PDF (`.pdf`), and Word (`.docx`) report downloads.
- **ThemeSelector.jsx** : Visual control menu for toggling system-wide color palettes and background gradient themes.
- **Avatar.jsx** : Avatar selector component rendering guest participant avatars for live lobby sessions.

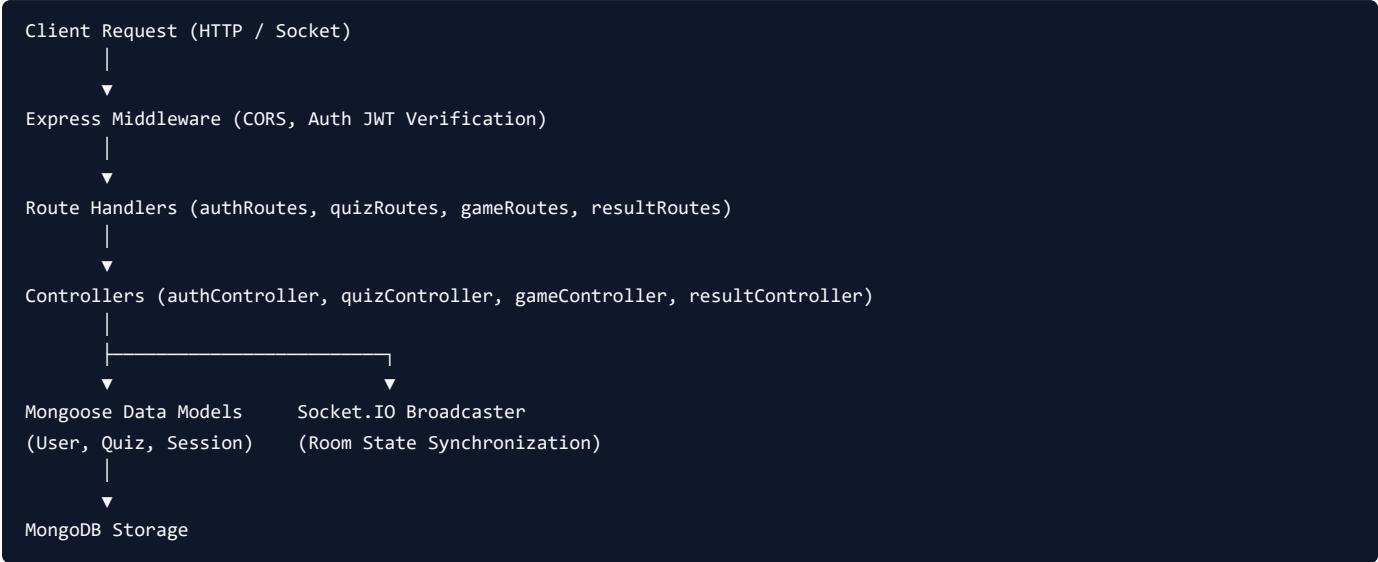
20. State Management

QuizForge utilizes a combined state architecture:

1. **Global React Context API** (`GameContext.jsx`): Holds active session state, room PIN, player credentials, and Socket.IO connection instance.
2. **Global Theme Context** (`ThemeContext.jsx`): Stores current user color scheme and visual theme parameters.
3. **Server Query Caching (TanStack React Query)**: Manages async fetching, caching, invalidation, and refetching of REST API data (quizzes, analytics reports).
4. **Local Component State** (`useState`): Manages ephemeral UI states like modal visibility, form fields, active question indices, and countdown timers.

21. Backend Architecture

The Express 5 backend enforces a standard layered architecture:



22. Security Analysis

22.1 Security Controls Identified

- **Password Hashing:** Passwords and security answers are hashed using `bcryptjs` (salt rounds: 10).
- **Stateless Token Security:** JWTs are signed with HMAC SHA256 (`JWT_SECRET`).
- **Protected Routes:** Enforced via `authMiddleware.js` verifying bearer token signatures prior to executing sensitive endpoints.

22.2 Potential Risks & Recommendations

1. **CORS Configuration:** `server/index.js` currently uses `origin: '*'`. In production, restrict this strictly to `process.env.FRONTEND_URL`.
2. **Rate Limiting:** Implement rate limiting middleware (e.g., `express-rate-limit`) on `/api/auth/login` and `/api/game/join` to prevent brute-force attacks.
3. **Input Validation:** Enforce payload schema validation (e.g., via `zod` or `joi`) to prevent NoSQL query injection payloads.

23. Performance Analysis

- **Socket Room Isolation:** Sockets subscribe strictly to room channels matching their specific game PIN (`room_<PIN>`), eliminating global payload overhead.
- **Database Queries:** Questions and player answers are stored as embedded arrays within `Quiz` and `GameSession` documents, ensuring high-speed single-read query performance.
- **Bundle Optimization:** Frontend is compiled via Vite 8 with code-splitting and asset minification.

24. Deployment Documentation

24.1 Deployment Setup

Backend (Render / Railway / VPS)

1. Environment variables required: `MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRE`, `PORT`, `FRONTEND_URL`.
2. Start command: `npm --prefix server install && npm --prefix server run start`.
3. Server entry point: `server/index.js`.

Frontend (Vercel / Netlify)

1. Environment variables required: `VITE_API_URL`.
2. Build command: `npm --prefix client install && npm --prefix client run build`.
3. Output directory: `client/dist`.
4. Single Page Application rewrite rules are configured in `client/vercel.json`.

25. Third-Party Integrations

- **Nodemailer:** Transports email verification and reset tokens over SMTP.
- **QRCode (`qrcode`):** Renders base64 data URIs representing game join links.
- **Socket.IO:** Facilitates real-time WebSocket messaging.
- **Canvas Confetti:** Triggers visual particle animations upon quiz completion.
- **Document Exporters (`xlsx` , `jspdf` , `jspdf-autotable` , `docx`):** Generate client-side report downloads.

26. Reports & Analytics

26.1 Analytics Engine Capabilities

- **Accuracy Metrics:** Percentage of overall correct answers submitted by players.
- **Per-Question Analytics:** Item analysis showing option selection distribution across all players.
- **Export Utilities:**
 - `.xlsx` : Excel workbook containing Player Leaderboard tab and Detailed Answer Matrix tab.
 - `.pdf` : Formatted PDF report document containing session metrics and winner podium summary.
 - `.docx` : Editable Word document report for printing and archiving.

27. Hidden & Experimental Features

- **Network Tunneling Configurations:** `client/src/tunnel-config.json` and `client/src/ngrok-config.json` contain localtunnel and ngrok configuration parameters for exposing local dev servers to external mobile devices during testing.
- **Image Helper Routes:** `server/routes/imageRoutes.js` includes proxy endpoints for retrieving and converting remote image URLs for canvas and PDF compatibility.

28. Complete Project Workflow

```
[1. Host Registration] -> [2. Create Quiz with 4 Options] -> [3. Launch Game Session]
                                                                |
[6. Export PDF/Excel] <- [5. Live Quiz & Leaderboard] <- [4. 6-Digit PIN / QR Join]
```


29. Mermaid Diagrams

29.1 Full System Architecture Diagram

```
graph TD
    Client[React 19 SPA Client]
    Express[Express 5 Server Node.js]
    Socket[Socket.IO Event Engine]
    Mongo[(MongoDB Database)]

    Client -->|HTTP REST APIs| Express
    Client <-->|WebSocket Connection| Socket
    Express -->|Mongoose ODM| Mongo
    Socket -->|State Persistence| Mongo
```

29.2 Realtime Live Quiz Socket Event Flow

```
sequenceDiagram
    autonumber
    actor Host
    actor Player
    participant SocketServer
    participant DB

    Host->>SocketServer: Emit `host-start-game` (PIN)
    SocketServer->>DB: Update GameSession status to 'active'
    SocketServer-->>Player: Broadcast `show-question` (Question data, timer)

    Player->>SocketServer: Emit `submit-answer` (PIN, Option, Speed)
    SocketServer->>SocketServer: Compute Points & Update Player Score
    SocketServer-->>Host: Broadcast `player-answered-count`

    SocketServer-->>Player: Broadcast `question-ended` + `show-answer-result`
    SocketServer-->>Host: Broadcast `update-leaderboard`
```

30. Statistics

- **Frontend SPA Pages:** 17 React route components
- **Frontend Components:** 24 Reusable components (including 11 landing sections)
- **Frontend Services & Contexts:** 7 API/Socket services, 2 React Context providers
- **Backend Controllers:** 5 Active Express controllers
- **Backend Routes:** 6 Router modules
- **Mongoose Models:** 6 Database schemas (`User` , `Quiz` , `GameSession` , `Result` , `Plan` , `FAQ`)
- **REST API Endpoints:** 22 Formally declared endpoints

31. Dependency Matrix

Feature / Goal	Frontend Route	Client Service	Express Route Endpoint	Controller Handler	Mongoose Model
User Sign Up	/register	authService.js	POST /api/auth/register	registerUser	User.js
User Login	/login	authService.js	POST /api/auth/login	loginUser	User.js
Create Quiz	/quiz/create	quizService.js	POST /api/quiz/create	createQuiz	Quiz.js
Launch Game	/dashboard	gameService.js	POST /api/game/create	createGameSession	GameSession.js
Join Game	/join	gameService.js	POST /api/game/join	joinGameSession	GameSession.js
Submit Answer	/live:pin	socketService.js	WebSocket submit-answer	socket/index.js	GameSession.js
Save Results	/final-result/:pin	resultService.js	POST /api/result/save	saveResult	Result.js
View Analytics	/results/:id	resultService.js	GET /api/result/:id	getResultBySessionId	Result.js

32. Final Deliverables & Maintenance Guide

32.1 Developer Onboarding & Quickstart

```
# 1. Install all monorepo dependencies
npm install
npm --prefix server install
npm --prefix client install

# 2. Configure Environment Files
Copy-Item server/.env.example server/.env

# 3. Launch Development Server
npm run dev:full
```

32.2 Maintenance Checklist

1. **Database Backups:** Regularly back up MongoDB using standard CLI tools:

```
mongodump --uri="<MONGO_URI>" --out=/backups/quizforge-$(date +%F)
```

2. **Environment Key Security:** Ensure `JWT_SECRET` and `MONGO_URI` are never committed to public repositories.

3. **Log Monitoring:** Monitor backend logs via PM2 or hosting provider dashboards:

```
pm2 logs server
```